



## BITCOIN CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

### 01. OVERVIEW & SETUP

Installation

Bitcoin Blockchain/Web3 technology

```
# Install
npm install bitcoin
# Or: pip install / gem install / go get
```

Check official documentation at docs.bitcoin.com  
Verify installation before proceeding

### 04. CONFIGURATION

Workflow

```
# config.yaml or config.json
bitcoin:
  host: localhost
  port: 8080
  timeout: 30s
```

Use environment variables for secrets

```
export BITCOIN_SECRET=my-secret
```

### 02. CORE CONCEPTS

Architecture

Key abstraction: understand the primary model  
Configuration: define behavior through config/code  
Integration: connects with existing systems

```
# Initialize
const client = new Bitcoin({ /* config */ })
```

### 05. BASIC OPERATIONS

CRUD API

```
# Create / write
client.create({ name: 'example' })
# Read / query
const result = await client.find({ filter: {} })
# Update
await client.update(id, { field: newValue })
# Delete
await client.delete(id)
```

### 03. QUICK START

YAML / Config

```
# Minimal working example
const app = new Bitcoin()
app.configure({ ... })
await app.start()
console.log('Bitcoin is running')
```

See official quickstart guide for language-specific setup

### 06. INTEGRATION

Integration

Integrates with common frameworks and tools

```
# Express.js integration example
app.use(BitcoinMiddleware({ config }))
```

SDK available for: Node.js, Python, Java, Go, .NET  
REST API available for language-agnostic integration  
Check community-maintained integrations



# SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

## 07. SECURITY

Hardening

Always use authentication and authorization  
Encrypt data at rest and in transit (TLS/SSL)

```
# Enable TLS
tls: { cert: ./cert.pem, key: ./key.pem }
```

Rotate credentials and API keys regularly  
Follow security best practices in official docs

## 10. TESTING

Unit Tests

```
# Unit test example
describe('Bitcoin', () => {
  it('should work', async () => {
    const result = await client.doSomething()
    expect(result).toBeDefined()
  })
})
```

Use mocks for external dependencies in unit tests

## 08. MONITORING

Observability

Expose health check endpoint

```
# Health check
GET /health { status: 'ok', uptime: 1234 }
```

Monitor key metrics: latency, throughput, error rate  
Set up alerts for anomalies  
Log requests with structured logging (JSON format)

## 11. CLI REFERENCE

Commands

```
bitcoin --help      # all commands
bitcoin version     # version info
bitcoin config       # configuration
bitcoin status       # current status
bitcoin logs         # view logs
```

## 09. PERFORMANCE TIPS

Optimization

Use connection pooling for network resources  
Enable caching for frequently accessed data  
Batch operations to reduce round trips

```
# Async/parallel processing
await Promise.all([op1(), op2(), op3()])
```

Profile before optimizing measure, then improve

## 12. COMMON GOTCHAS

Warning

Read the changelog before upgrading major versions  
Check resource limits (memory, connections, rate limits)  
Implement graceful shutdown to avoid data loss  
Keep dependencies updated for security patches  
Test in staging environment before deploying to production